Data Science (COL2010) — Homework 2 (V2)

Module 2: Data Cleaning and Pre-processing

Total: 100 marks

Learning objective

This homework follows one dirty relational database from diagnosis to repair. You will first learn the library functions used for profiling, comparison, missing-data treatment, outlier detection, and transformation. You will then combine those tools into complete cleaning procedures for the same database: profile the data, standardize fields, generate duplicate candidates, resolve entities, diagnose and repair missing values, detect unusual records, transform features, and produce an auditable cleaned dataset.

Relevant Libraries and Imports

Useful Libraries and Functions:

To help you get started, the following Python libraries and functions are likely to be useful for HW2: pandas for data manipulation (DataFrame, Series, read_csv, value_counts, isna, dropna, groupby), NumPy for numerical operations and arrays, SciPy for distance and statistical calculations (scipy.spatial.distance, zscore), scikit-learn for imputation, scaling, encoding, and outlier detection (SimpleImputer, KNNImputer, IterativeImputer, StandardScaler, MinMaxScaler, RobustScaler, PowerTransformer, OneHotEncoder, LocalOutlierFactor, MinCovDet), and RapidFuzz for record/string similarity (Levenshtein, JaroWinkler). You are encouraged to refer to the relevant library documentation and use these functions where appropriate.

Files

Run

```
python setup.py
```

before starting. The setup program creates:

- `customers.csv`
- `transactions.csv`
- `contacts.csv`
- `module2_parameters.json`
- `module2_inputs.npz`
- `module2_gold.npz`

Complete `skeleton-solutions.py` and submit it as `solutions.py`. The reference `solutions.py` will be released later.

Loading data and parameters

The function signature tells you what a function receives. This section tells you where those arguments come from. Do not invent a parameter value when a supplied value is named.

```
import json
import numpy as np
import pandas as pd

customers = pd.read_csv("customers.csv")
transactions = pd.read_csv("transactions.csv")
```

```
contacts = pd.read_csv("contacts.csv")

with open("module2_parameters.json", "r") as f:
    params = json.load(f)

inputs = np.load("module2_inputs.npz", allow_pickle=False)

# Exploration only:
gold = np.load("module2_gold.npz", allow_pickle=False)
```

There are four possible sources of function arguments:

1. **CSV tables:** the dirty relational data.
2. **`module2_parameters.json`:** named scalar, list, dictionary, and threshold parameters such as the city dictionary, entity-resolution weights, entity thresholds, imputation settings, outlier settings, and feature-column lists.
3. **`module2_inputs.npz`:** student-visible NumPy arrays that are inconvenient to store in JSON, including the Part-I practice matrix, Task 20 evaluation data/mask, and Task 23 train/test indices.
4. **The autograder or your own test cases:** the small Part-I functions are general-purpose functions. The autograder calls them on hidden inputs; for local testing, use the supplied examples named in each task or construct additional small cases yourself.

Named parameter map for the supplied homework instance

Supplied key	Main use
`params["valid_age_range"]`	Task 13
`params["valid_email_regex"]`	Task 13
`params["city_aliases"]`	Task 14
`params["entity_weights"]`	Task 17
`params["entity_thresholds"]["lower"]` and `["upper"]`	Task 17
`params["missingness_examples"]`	Task 19
`params["imputation_k"]` and `params["imputation_random_state"]`	Task 20
`params["outlier_contamination"]`, `params["lof_neighbors"]`, and `params["outlier_numeric_columns"]`	Task 22
`params["model_numeric_columns"]` and `params["model_categorical_columns"]`	Task 23
`inputs["imputation_complete_subset"]` and `inputs["imputation_evaluation_mask"]`	Task 20
`inputs["train_indices"]` and `inputs["test_indices"]`	Task 23

How to call and test your functions

Function-parameter rule. Each function must use the arguments that are passed to it. The values generated by `setup.py` are the values for your local homework instance; they are *not* constants that may be hard-coded into your solution. The autograder may call the same function several times with different valid tables, arrays, column names, thresholds, weights, values of k, train/test indices, or other parameters.

A broad local test set

The generated database contains identifiers, numerical values, categorical values, dates, free-form strings, and fields with missing values. You should test general-purpose functions on a reasonably broad selection of these types. The following lists are **recommended local tests**; the hidden autograder may use other valid values.

```
profile_tests = {
    "customers": [
        "age",
        "annual_income",
        "household_size",
        "city",
        "name",
        "phone",
        "email",
        "dob",
        "signup_date",
    ],
    "transactions": [
        "amount",
        "items",
        "discount",
        "channel",
        "merchant_city",
        "transaction_date",
        "customer_id",
    ],
    "contacts": [
        "name",
        "phone",
        "email",
        "city",
        "last_verified",
        "source",
        "customer_id",
    ],
}

frames = {
    "customers": customers,
    "transactions": transactions,
    "contacts": contacts,
}

for table_name, columns in profile_tests.items():
    frame = frames[table_name]
    for column in columns:
        result = profile_column(frame, column)
        print(table_name, column, result)
```

This does *not* mean that every statistic is meaningful for every data type. One purpose of testing across several types is to discover which parts of a general profile are defined and useful for numeric, categorical, date-like, string, and identifier columns.

Examples for the other parameterized functions

The same principle applies throughout the homework: vary the arguments when testing your code.

```
# Task 2: categorical-frequency profiling
for frame, column in [
```

```
    (customers, "city"),
    (transactions, "channel"),
    (transactions, "merchant_city"),
    (contacts, "source"),
    (contacts, "city"),
]:
    print(top_category_frequencies(frame, column, k=10))

# Task 3: numerical distance -- use several row pairs
X = inputs["toolbox_numeric_X"]
for i, j in [(0, 1), (0, 5), (10, 20), (25, 50)]:
    print(numeric_distances(X[i], X[j]))

# Task 4: binary similarities
print(binary_similarities(
    inputs["toolbox_binary_x"],
    inputs["toolbox_binary_y"],
))

# Tasks 5--7: try several kinds of string pairs
string_pairs = [
    ("Ananya Rao", "Annya Rao"),
    ("Ananya Rao", "A. Rao"),
    ("Bengaluru", "Bangalore"),
    ("98765 43210", "+91-98765-43210"),
    ("maya@example.com", "maya@example.com"),
    ("Ananya Rao", "Vikram Sen"),
]

for a, b in string_pairs:
    print(a, b, levenshtein_pair(a, b), jaro_winkler_pair(a, b))

# Task 9: test more than one k
for k in [1, 3, 5, 9, 15]:
    X_knn = knn_imputation_library(
        inputs["toolbox_numeric_X"],
        k,
    )

# Task 11: vary the LOF neighbourhood size
for n_neighbors in [5, 10, 20, 30]:
    scores = multivariate_outlier_scores(
        inputs["toolbox_numeric_X"],
        n_neighbors,
    )

# Task 14: pass the supplied dictionary -- do not hard-code it
customers_std = standardize_customer_records(
    customers,
    params["city_aliases"],
)

# Task 17: supplied parameters for the main homework instance
pair_decisions = classify_candidate_pairs(
    comparisons,
    params["entity_weights"],
    params["entity_thresholds"]["lower"],
    params["entity_thresholds"]["upper"],
)

# Also test Task 17 with another valid threshold pair
```

```
pair_decisions_test = classify_candidate_pairs(
    comparisons,
    params["entity_weights"],
    0.50,
    0.75,
)

# Task 19: setup supplies several target/conditioning configurations
for mechanism, spec in params["missingness_examples"].items():
    report = missingness_diagnostics(
        customers,
        spec["target"],
        spec["conditioning"],
    )
    print(mechanism, report)

# Task 20
imputation_report = evaluate_imputation_methods(
    inputs["imputation_complete_subset"],
    inputs["imputation_evaluation_mask"],
    params["imputation_k"],
    params["imputation_random_state"],
)

# Task 22: vary detector parameters
customer_numeric = customers[params["outlier_numeric_columns"]]
for contamination in [0.01, 0.02, 0.05]:
    for n_neighbors in [10, 20, 30]:
        result = detect_database_outliers(
            customer_numeric,
            transactions["amount"],
            contamination,
            n_neighbors,
        )

# Task 23: supplied split for the main homework instance
model_data = build_model_ready_matrices(
    customers,
    params["model_numeric_columns"],
    params["model_categorical_columns"],
    inputs["train_indices"],
    inputs["test_indices"],
)
```

The generated dirty database

The setup program is deterministic for the published seed and generates approximately 14,000–15,000 tuples in total.

Table	Columns and intended role
`customers`	approximately 6,000–6,500 rows: `customer_id`, `name`, `dob`, `age`, `city`, `phone`, `email`, `annual_income`, `household_size`, `signup_date`.
`transactions`	approximately 5,000–5,500 rows: `transaction_id`, `customer_id`, `transaction_date`, `amount`, `channel`, `merchant_city`, `items`, `discount`.
`contacts`	approximately 3,000 rows: `contact_id`, `customer_id`, `name`, `phone`, `email`, `city`, `last_verified`, `source`.

The generator deliberately injects the following defects.

1. **Near-duplicate values.** Examples include `Bangalore/Bengaluru`, case and whitespace variants, punctuation and country-code differences in phone numbers, and typographical errors in names and e-mail addresses.
2. **Near-duplicate entities.** The same real customer may occur more than once, for example `Ananya Rao`, `A. Rao`, and `Ananya R.`, with different subsets of fields agreeing. The gold file stores the true entity identifier for every generated customer-like record.
3. **Missing data.** Different columns contain MCAR, MAR, and MNAR missingness. The gold file stores the clean values and the mechanism masks. The student-visible tables do not reveal which individual cells were generated under which mechanism.
4. **Outliers.** Include both univariate and multivariate outliers. Some are deliberate errors; some are unusual but valid observations. The homework therefore asks students to *flag* candidate outliers rather than automatically delete every unusual record.

Rules

1. Use Python 3.10 or later.
2. Permitted packages: `numpy`, `pandas`, `scipy`, `scikit-learn`, `rapidfuzz`, and `matplotlib`.
3. High-level library functions are encouraged in Part I. Part II asks you to combine them into specified cleaning procedures.
4. Do not change any function name, argument order, or return type.
5. The autograder imports your functions directly and tests them on hidden inputs with the same contracts. Do not hard-code row counts or generated answers.
6. Unless explicitly stated, input data frames must not be modified in place.
7. Missing values must remain missing unless a task explicitly asks you to repair them.
8. For every returned list of row IDs or row-pairs, sort deterministically as specified.
9. The **Exploration** items are for learning only. They are not used by the code autograder.

Part I — Using the Data-Cleaning Toolbox [30 marks]

These tasks are intentionally small. Their purpose is to make you comfortable with standard functions that will be reused in Part II.

Task 1: Column profiling with pandas [2 marks]

Purpose: Use standard dataframe summaries rather than writing every statistic from scratch.

Return the non-missing count, missing count, number of distinct non-missing values, minimum, maximum, mean, median, first quartile, and third quartile. Compute minimum and maximum normally using the non-missing values. Compute mean, median, first quartile, and third quartile only for numerical columns; for non-numerical columns, return np.nan for these four statistics.

Inputs. The autograder supplies `frame` and `column`. For local testing, use `customers` and one of `params["toolbox"]["profile_columns"]`.

Strict Python interface

```
def profile_column(frame:  pd.DataFrame, column:  str) -> dict:
```

Autograder output. Return the keys `non_missing`, `missing`, `distinct`, `min`, `max`, `mean`, `median`, `q1`, and `q3`.

Exploration (not autograded). Use this function on `annual_income`, `city`, and `phone`. Explain why the same profile fields do not carry the same meaning for all three data types.

Task 2: Frequency profile and rare categories [2 marks]

Purpose: Use `value_counts` to expose fragmented categorical domains.

Return a two-column data frame `value,count` for the k most frequent non-missing values, ordered by decreasing count and then lexicographically.

Inputs. The autograder supplies `frame`, `column`, and `k`. For local testing, use any generated table and `params["toolbox"]["frequency_k"]`.

Strict Python interface

```
def top_category_frequencies(frame:  pd.DataFrame, column:  str, k:  int) -> pd.DataFrame:
```

Autograder output. The returned columns must be exactly `value` and `count`.

Exploration (not autograded). Apply this to `city`, `channel`, and `source`. Identify variants that look like formatting differences and variants that require domain knowledge before merging.

Task 3: Euclidean and Manhattan distance [2 marks]

Purpose: Use `scipy.spatial.distance`.

Return the Euclidean and Manhattan distances between two numerical vectors.

Inputs. The autograder supplies `x` and `y`. For local testing, use two rows from `inputs["toolbox_numeric_X"]`.

Strict Python interface

```
def numeric_distances(x:  np.ndarray, y:  np.ndarray) -> tuple[float, float]:
```

Autograder output. Return `(euclidean, manhattan)` as Python floats.

Exploration (not autograded). Compare distances before and after multiplying the income feature by 10^5. Explain why Lecture 5 scaling changes nearest-neighbour behaviour.

Task 4: Jaccard and cosine similarity [2 marks]

Purpose: Use standard similarity functions on binary vectors.

Return Jaccard similarity and cosine similarity for two binary vectors. If both input vectors are all zeros, define cosine similarity as 1.0. If exactly one vector is all zeros, define cosine similarity as 0.0.

Inputs. The autograder supplies `x` and `y`. For local testing, use `inputs["toolbox_binary_x"]` and `inputs["toolbox_binary_y"]`.

Strict Python interface

```
def binary_similarities(x:  np.ndarray, y:  np.ndarray) -> tuple[float, float]:
```

Autograder output. Return `(jaccard, cosine)`.

Exploration (not autograded). Construct a sparse pair with many shared zeroes and explain why Jaccard is often preferable to the simple matching coefficient for presence/absence data.

Task 5: Levenshtein distance and normalized similarity [2 marks]

Purpose: Use `rapidfuzz.distance.Levenshtein`.

For two strings, return the Levenshtein edit distance and normalized similarity in $[0, 1]$. Compare the strings as supplied; do not perform additional normalization such as lowercasing or whitespace removal.

Inputs. The autograder supplies the two strings. For local testing, use name/e-mail values from `customers.csv` and controlled examples of your own.

Strict Python interface

```
def levenshtein_pair(a:  str, b:  str) -> tuple[int, float]:
```

Autograder output. Return `(distance, normalized_similarity)`.

Exploration (not autograded). Compare typographical errors, abbreviations, and completely different names. Which cases does edit distance handle poorly?

Task 6: Jaro–Winkler similarity [2 marks]

Purpose: Use `rapidfuzz.distance.JaroWinkler`.

Return the Jaro–Winkler similarity between two strings. Compare the strings as supplied; do not perform additional normalization such as lowercasing or whitespace removal.

Inputs. The autograder supplies the two strings. For local testing, use name values from `customers.csv` or the examples in the Exploration paragraph.

Strict Python interface

```
def jaro_winkler_pair(a:  str, b:  str) -> float:
```

Autograder output. Return one float in $[0, 1]$.

Exploration (not autograded). Compare `Ananya Rao`/`Annya Rao`, `Ananya Rao`/`A. Rao`, and two unrelated names. Relate the behaviour to the prefix-sensitive design discussed in class.

Task 7: Token-set Jaccard for strings [2 marks]

Purpose: Tokenize, normalize, and compare unordered word sets.

Lower-case, split on whitespace, discard empty tokens, treat the resulting tokens as sets, and return $|A \cap B|/|A \cup B|$. Define the similarity of two empty sets to be 1.

Inputs. The autograder supplies the two strings. For local testing, construct short multi-token name/address examples yourself.

Strict Python interface

```
def token_set_jaccard(a:  str, b:  str) -> float:
```

Autograder output. Return one float in $[0, 1]$.

Exploration (not autograded). Try person names and multi-word addresses. Give one example where token order should matter and one where it should not.

Task 8: Simple imputation with scikit-learn [3 marks]

Purpose: Learn the standard `SimpleImputer` interface.

Fit both a mean imputer and a median imputer to the supplied matrix and return the two completed matrices.

The returned matrices must have the same number of rows and columns as X, including when a column contains only missing values.

Inputs. The autograder supplies `X`. For local testing, use `inputs["toolbox_numeric_X"]`.

Strict Python interface

```
def simple_imputation_library(X: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
```

Autograder output. Return `(mean_imputed, median_imputed)` with the original shape.

Exploration (not autograded). On the hidden-clean demonstration data supplied by setup, compare RMSE and variance after mean and median imputation.

Task 9: KNN imputation with scikit-learn [3 marks]

Purpose: Use `sklearn.impute.KNNImputer`.

Apply KNN imputation with the supplied k. The supplied k is a positive integer smaller than the number of rows in X. X contains at least one observed value in every column.

Inputs. The autograder supplies `X` and `k`. For local testing, use `inputs["toolbox_numeric_X"]` and `params["toolbox"]["knn_k"]`.

Strict Python interface

```
def knn_imputation_library(X: np.ndarray, k:  int) -> np.ndarray:
```

Autograder output. Return a fully imputed matrix with the original shape.

Exploration (not autograded). Plot masked-value RMSE for $k \in \{1, 3, 5, 9, 15\}$. Repeat after rescaling one feature by 1000 and explain the result.

Task 10: Univariate outlier helpers [3 marks]

Purpose: Use SciPy/NumPy functions to reproduce the three detectors studied in class.

Return ordinary z-scores, modified z-scores based on MAD, and the lower/upper Tukey fences. Missing values must remain unflagged and may be represented by `np.nan` in score arrays. Compute all statistics using the non-missing values.

If the standard deviation is zero, return a score of 0.0 for all observed values. If the MAD is zero, return a modified z-score of 0.0 for all observed values. Missing values must remain np.nan.

Inputs. The autograder supplies `x`. For local testing, use one column from `inputs["toolbox_numeric_X"]`.

Strict Python interface

```
def univariate_outlier_helpers(x:  np.ndarray) -> dict:
```

Autograder output. Return keys `z`, `modified_z`, `iqr_lower`, and `iqr_upper`.

Exploration (not autograded). Demonstrate masking by inserting several extreme values into an otherwise regular sample.

Task 11: Mahalanobis and Local Outlier Factor [3 marks]

Purpose: Learn two ready-made multivariate anomaly tools.

Median-impute the supplied matrix. Compute a Mahalanobis distance from the sample mean and covariance, and a Local Outlier Factor score using the supplied number of neighbours.

n_neighbors must satisfy $2 \leq n_neighbors < n$, where n is the number of rows in X.

Inputs. The autograder supplies `X` and `n_neighbors`. For local testing, use `inputs["toolbox_numeric_X"]` and `params["toolbox"]["lof_neighbors"]`.

Strict Python interface

```
def multivariate_outlier_scores(X: np.ndarray, n_neighbors:  int) -> dict:
```

Autograder output. Return keys `mahalanobis` and `lof`; both are length-n arrays where larger returned values mean "more unusual."

Exploration (not autograded). Construct two clusters of different density plus one local anomaly. Compare the global Mahalanobis ranking with the local LOF ranking.

Task 12: Scaling and power transforms [4 marks]

Purpose: Use standard scaler/transformer objects and inspect their effect.

Median-impute the supplied numerical matrix, then return Min–Max scaled, z-score standardized, robust-scaled, and Yeo–Johnson transformed versions. For the Yeo–Johnson transformation, do not additionally standardize the transformed values.

Inputs. The autograder supplies `X`. For local testing, use `inputs["toolbox_numeric_X"]`.

Strict Python interface

```
def transformation_library_bundle(X: np.ndarray) -> dict:
```

Autograder output. Return keys `minmax`, `zscore`, `robust`, and `yeojohnson`, all with the original shape.

Exploration (not autograded). Add one extreme value and show how it changes Min–Max, z-score, and robust scaling. For a skewed positive feature, compare the original, log-transformed, and Yeo–Johnson distributions.

Part II — Cleaning the Generated Database [70 marks]

All tasks in this part operate on the generated relational database. They are deliberately more integrated: most require several operations and, where appropriate, one or more of the library tools from Part I.

Task 13: Build a baseline data-quality profile [4 marks]

Purpose: Measure the database before changing it.

Construct a database-level profile with: row counts by table; missing counts by column; distinct counts by column; exact duplicate rows; invalid dates; ages outside the supplied domain; negative transaction amounts; invalid e-mail format; and foreign-key violations from `transactions.customer_id` and `contacts.customer_id` to `customers.customer_id`.

Inputs. Pass the loaded `customers`, `transactions`, and `contacts` data frames. Pass the entire loaded JSON dictionary as `params`; Task 13 uses `params["valid_age_range"]` and `params["valid_email_regex"]`.

Strict Python interface

```
def build_database_profile( customers:  pd.DataFrame, transactions:  pd.DataFrame, contacts:
pd.DataFrame, params:  dict, ) -> dict:
```

Autograder output. Return a dictionary with keys `table_rows`, `missing_counts`, `distinct_counts`, `exact_duplicate_counts`, `invalid_customer_rows`, `invalid_transaction_rows`, and `foreign_key_violations`. Every row-ID array must be sorted.

Exploration (not autograded). Create a compact profiling dashboard with missing-rate bars, city frequencies, an income histogram, a transaction-amount boxplot, and the top ten detected validation violations.

Task 14: Standardize and canonicalize customer fields [4 marks]

Purpose: Produce comparable values without yet deciding which rows are the same entity.

Apply the supplied rules: trim/collapse whitespace; Unicode-normalize; lower-case names and e-mails; remove punctuation from phone numbers and retain the final ten digits; canonicalize known city variants through the supplied dictionary; and standardize dates to ISO `YYYY-MM-DD`. Preserve missing values.

Inputs. Use `customers` as `frame`. Supply `params["city_aliases"]` as `city_aliases`.

Strict Python interface

```
def standardize_customer_records( frame:  pd.DataFrame, city_aliases:  dict[str, str], ) ->
pd.DataFrame:
```

Autograder output. Return the same rows and columns in the same order. The input frame must not be modified.

Exploration (not autograded). Report distinct-cardinality before and after standardization for name, city, phone, and e-mail. Explain why a reduction in cardinality is not automatically proof of better data.

Task 15: Generate candidate duplicate pairs with two blocking passes [6 marks]

Purpose: Avoid the $O(n^2)$ all-pairs comparison while protecting recall.

Use two blocking passes on standardized customer records: (1) same canonical city and same birth year; (2) same final four phone digits, ignoring missing phones. Return the union of both passes as unique pairs (i, j) with $i < j$.

Inputs. Use the standardized customer frame returned by Task 14 as `customers_std`. No new setup parameter is required.

Strict Python interface

```
def generate_candidate_pairs(customers_std:  pd.DataFrame) -> np.ndarray:
```

Autograder output. Return an integer array of shape $(m, 2)$, lexicographically sorted. For no candidates, return shape $(0, 2)$.

Exploration (not autograded). Using the gold duplicate pairs provided only for exploration, compute reduction ratio and pair completeness for each pass separately and for their union. Plot recall against candidate count.

Task 16: Compute a multi-field comparison vector [6 marks]

Purpose: Treat an entity decision as evidence across multiple fields, not as one string comparison.

For every candidate pair, compute: Jaro–Winkler on name; normalized Levenshtein similarity on e-mail; exact agreement on standardized phone; exact agreement on canonical city; and date-of-birth agreement. Missing comparison fields must produce `np.nan`, not zero.

Inputs. Use the standardized frame from Task 14 and the candidate-pair array returned by Task 15.

Strict Python interface

```
def compare_candidate_pairs( customers_std:  pd.DataFrame, pairs:  np.ndarray, ) ->
pd.DataFrame:
```

Autograder output. Return columns exactly `i`, `j`, `name_sim`, `email_sim`, `phone_agree`, `city_agree`, `dob_agree`, in the same pair order.

Exploration (not autograded). Plot match and non-match distributions for each field similarity using the gold labels. Which field is individually most discriminative?

Task 17: Score and classify record pairs [6 marks]

Purpose: Combine field evidence while distinguishing missing evidence from disagreement.

Given the comparison table and supplied field weights, calculate a weighted mean over only the available comparisons. Classify each candidate as `nonmatch`, `review`, or `match` using supplied lower and upper thresholds.

Inputs. Use the comparison table from Task 16.

Supply `params["entity_weights"]`, `params["entity_thresholds"]["lower"]`, and `params["entity_thresholds"]["upper"]`.

Strict Python interface

```
def classify_candidate_pairs( comparisons:  pd.DataFrame, weights:  dict[str, float],
lower_threshold:  float, upper_threshold:  float, ) -> pd.DataFrame:
```

Autograder output. Return columns exactly `i`, `j`, `score`, `decision`. Rows must remain in input order.

Exploration (not autograded). Sweep the upper threshold and plot precision, recall, and F1 against it. Then vary the review zone and count how many pairs are sent to manual review.

Task 18: Form entity clusters and construct golden records [8 marks]

Purpose: Turn accepted pair matches into coherent entities and preserve provenance.

Use only pairs classified as `match`. Form connected components over customer rows. For each component create one golden record: use the median for numerical values; for strings use the most frequent non-missing standardized value, breaking ties by longer string and then lexicographic order. Add `cluster_size` and `source_row_ids`.

Inputs. Use the standardized frame from Task 14 and the decision table from Task 17. No new setup parameter is required.

Strict Python interface

```
def resolve_customer_entities( customers_std:  pd.DataFrame, pair_decisions:  pd.DataFrame,
) -> tuple[np.ndarray, pd.DataFrame]:
```

Autograder output. Return `(cluster_labels, golden_records)`. Cluster labels must be consecutive integers starting at 0, ordered by the smallest original row index in each component.

Exploration (not autograded). Create an $A \leftrightarrow B \leftrightarrow C$ chain where A and C are not a direct match. Explain the transitivity issue. Inspect at least five generated golden records and identify where provenance is essential.

Task 19: Diagnose MCAR, MAR, and MNAR patterns [5 marks]

Purpose: Use observable association tests as evidence, but do not claim that MNAR is provable from observed data alone.

For a target column, create a missingness indicator. For each supplied observed conditioning column, return the difference in numerical mean or categorical missingness rate between rows where the target is observed and missing.

Inputs. Call the function once for each entry in `params["missingness_examples"]`. Use `customers` as `frame`; each entry supplies its `target` and `conditioning` list.

Strict Python interface

```
def missingness_diagnostics( frame:  pd.DataFrame, target_column:  str,
conditioning_columns:  list[str], ) -> pd.DataFrame:
```

Autograder output. Return columns exactly `conditioning_column`, `statistic`, `value`. Sort by `conditioning_column`.

Exploration (not autograded). Use the hidden mechanism masks from setup only in the exploration report to verify which generated columns are MCAR, MAR, and MNAR. Explain why the MNAR conclusion cannot be established from the dirty table alone.

Task 20: Evaluate three imputation strategies by masking [6 marks]

Purpose: Judge repairs only where the true value is known.

For the specified numerical columns, mask the supplied evaluation cells and compare: median imputation; KNN imputation; and iterative imputation. Return RMSE per column and overall RMSE for each method.

Inputs. Use `inputs["imputation_complete_subset"]` as `X_complete_subset`,

`inputs["imputation_evaluation_mask"]` as `evaluation_mask`, `params["imputation_k"]` as `k`, and `params["imputation_random_state"]` as `random_state`.

Strict Python interface

```
def evaluate_imputation_methods( X_complete_subset:  np.ndarray, evaluation_mask:
np.ndarray, k:  int, random_state:  int, ) -> pd.DataFrame:
```

Autograder output. Return columns exactly `method`, `column`, `rmse`. Include one extra row per method with `column="OVERALL"`. Sort by method then column.

Exploration (not autograded). For income, compare the clean, masked, and repaired distributions. Report both RMSE and variance distortion. Explain why the numerically best RMSE need not preserve every downstream property.

Task 21: Create missingness indicators and an imputation audit [4 marks]

Purpose: Do not erase the fact that a value was originally missing.

For every imputed column, append a Boolean `<column>_was_missing` indicator and create an audit table listing every changed cell.

Inputs. Use the dirty numerical frame before imputation as `original`, the corresponding repaired frame as `imputed`, and `list(inputs["numeric_columns"].astype(str))` as `columns`.

Strict Python interface

```
def build_imputation_audit( original:  pd.DataFrame, imputed:  pd.DataFrame, columns:
list[str], ) -> tuple[pd.DataFrame, pd.DataFrame]:
```

Autograder output. Return `(augmented, audit)`. The audit columns must be exactly `row_index`, `column`, `old_value`, `new_value`, sorted by row index then column.

Exploration (not autograded). Fit a simple downstream model with and without missingness indicators on the exploration target supplied by setup. Compare performance and interpret cautiously.

Task 22: Detect univariate and multivariate outliers [7 marks]

Purpose: Apply several detectors because each encodes a different assumption.

For customer numerical features and transaction amounts: flag univariate candidates using modified z-score and IQR; compute robust Mahalanobis distances for the selected customer features; and compute LOF scores. Do not delete any row.

Inputs. Construct `customers_numeric = customers[params["outlier_numeric_columns"]]`, use `transactions["amount"]` as `transaction_amounts`, and supply `params["outlier_contamination"]` and `params["lof_neighbors"]`.

Strict Python interface

```
def detect_database_outliers( customers_numeric:  pd.DataFrame, transaction_amounts:
pd.Series, contamination:  float, n_neighbors:  int, ) -> dict:
```

Autograder output. Return keys `customer_mad_flags`, `customer_iqr_flags`, `transaction_mad_flags`, `transaction_iqr_flags`, `mahalanobis_scores`, and `lof_scores`.

Exploration (not autograded). Plot a two-dimensional customer feature view. Mark records that are ordinary in each coordinate but unusual jointly. Compare detector overlap and inspect disagreements rather than assuming one detector is correct.

Task 23: Construct a leak-free model-ready feature matrix [7 marks]

Purpose: Apply the course preprocessing sequence to a realistic table.

Given pre-specified train/test row indices: fit numerical imputation and scaling on training rows only; fit categorical one-hot encoding on training rows only; apply the fitted transformations to both train and test; and construct two supplied engineered features after imputation. Unseen test categories must be handled without failure.

Inputs. Use `customers` as `frame`;

supply `params["model_numeric_columns"]` and `params["model_categorical_columns"]`; and use `inputs["train_indices"]` and `inputs["test_indices"]`. The two engineered features are described in `params["engineered_features"]`.

Strict Python interface

```
def build_model_ready_matrices( frame:  pd.DataFrame, numeric_columns:  list[str],
categorical_columns:  list[str], train_indices:  np.ndarray, test_indices:  np.ndarray, )
-> dict:
```

Autograder output. Return keys

`X_train`, `X_test`, `feature_names`, `train_medians`, and `train_scaler_center`. The two matrices must have the same number and order of columns.

Exploration (not autograded). Deliberately fit preprocessing on the full dataset and compare with the correct split-first pipeline over multiple random splits. Explain why leakage remains invalid even when the numerical difference in one split is small.

Task 24: End-to-end cleaning audit [7 marks]

Purpose: Show that cleaning is a sequence of measurable, auditable decisions.

Run your previous functions to produce one final audit summary. The function must *not* repeat the full cleaning itself; it should consume the outputs of the earlier stages.

Inputs. Every argument is an earlier output: Task 13 `baseline_profile`; Task 15 `candidate_pairs`; Task 17 `pair_decisions`; Task 18 `golden_records`; Task 21 `imputation_audit`; Task 22 `outlier_results`; and Task 23 `model_matrix_info`. No new setup parameter is introduced.

Strict Python interface

```
def summarize_cleaning_audit( baseline_profile:  dict, candidate_pairs:  np.ndarray,
pair_decisions:  pd.DataFrame, golden_records:  pd.DataFrame, imputation_audit:
pd.DataFrame, outlier_results:  dict, model_matrix_info:  dict, ) -> dict:
```

Autograder output. Return exactly these keys:

`baseline_rows`, `candidate_pair_count`, `match_pair_count`, `review_pair_count`, `golden_entity_count`, `imputed_cell_count`, `flagged_outlier_count`, and `model_feature_count`.

Exploration (not autograded). Write a one-page decision log containing at least five concrete generated records: one canonicalization decision, one duplicate merge, one uncertain pair sent for review, one imputed value, and one outlier retained for human investigation. For each, state the evidence, action, and provenance retained.

Autograder conventions

1. Data-frame column names must match the specification exactly.
2. Unless otherwise specified, row indices refer to the current input frame's integer positional indices, not `customer_id`.
3. Floating-point values are checked with tolerance; do not round inside returned arrays.
4. Empty pair outputs must have shape $(0, 2)$ and integer dtype.
5. Where a task returns scores, higher values must consistently mean "more similar" or "more unusual" as explicitly stated.
6. Randomized estimators must use the supplied `random_state`.
7. Functions must handle hidden data with the same schema but different generated values and different defect locations.